API. The code implements a Spark filter that can be included before any API route definition in place of the existing `requirePermission` filters. The actual implementation of the ABAC permission check is left abstract for now; you will develop implementations in the next sections. The code collects attributes into the four attribute categories described above by examining the Spark `Request` object and extracting the username and any groups populated during authentication. You can include other attributes, such as the current time, in the environment properties. Extracting these kind of environmental attributes makes it easier to test the access control rules because you can easily pass in different times of day in your tests. If you're using JWTs (chapter 6), then you might want to include claims from the JWT Claims Set in the subject attributes, such as the issuer or the issued-at time. Rather than using a simple `boolean` value to indicate the decision, you should use a custom `Decision` class. This is used to combine decisions from different policy rules, as you'll see in section 8.3.1.

Listing 8.9 Gathering attribute values

```
package com.manning.apisecurityinaction.controller;

import java.time.LocalTime;
import java.util.Map;

import spark.*;

import static spark.Spark.halt;

public abstract class ABACAccessController {
```